Aerrand Developer API Keys — Technical Requirements Document

Prepared for the University of Windsor Master of Applied Computing Industry Partnership Program — Aerrand Technologies Inc.

Document	TRD — Developer API Keys & Public Aerrand API v1
Version	1.0 (draft for review)
Date	2026-08-29
Status	Proposed
Codebase	`Aerrand_backen_infra` (monorepo: `aerrand_api` Django, `aerrand_customer_backend` / `aerrand_driver_backend` Node, `aerrand_business` / `aerrand_admin` SPAs, Caddy gateway)
Related specs	Aerrand Business Pricing Engine Spec v2.0 (`order/business_pricing.py`), Business Cancellation & Modification Policy v2.0 (`order/business_cancellation.py`), Vehicle Assignment & Cargo Capacity Spec v1.0, ADR-0001 account/profile/delivery split

Table of contents

1 Purpose and scope
2 Goals and non-goals
3 Current-state analysis (as built)
4 Target architecture
5 Functional requirements
6 Public API specification (v1)
7 Data model
8 Non-functional requirements
9 Security requirements
10 Implementation plan — codebase touch points
11 Configuration and deployment
12 Testing and acceptance criteria
13 Rollout plan
14 Open questions and decisions required

- Appendix A — Glossary
- Appendix B — cURL quickstart
- Appendix C — Status mapping
- Appendix D — Environment variable reference

1. Purpose and scope

External businesses (e-commerce stores, pharmacies, restaurants, 3PLs, ERP/WMS integrators) need to create, pay for, track and cancel **aerrands** (Aerrand Business deliveries) programmatically, without a human clicking through `business.aerrand.com`.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

This document specifies:

- **Developer API keys**: issuance, storage, scoping, rotation, revocation and auditing of long-lived machine credentials owned by an approved Aerrand Business account.
- **Public Aerrand API v1**: a versioned, key-authenticated HTTP API (`https://api.aerrand.com/v1/*`) for quotes, aerrand creation (single and multi-stop), reads, tracking, cancellation, wallet balance and webhooks.
- The **server-side changes** required in the existing Django API (`aerrand_api`), the Node driver backend, both SPAs, Caddy and Docker Compose to support the above safely.

Out of scope for v1 (see §2): Aerrand Guard (escrow/vendor KYC) orders, OAuth/partner multi-tenant apps, invoicing/postpaid billing, customer-app (aerrandweb / mobile) APIs.

---

2. Goals and non-goals

2.1 Goals

#	Goal	Success signal
G1	An approved business can mint an API key in the dashboard and create a paid, driver-visible aerrand with **one HTTP call**.	Time-to-first-aerrand from key creation < 10 minutes following the docs.
G2	Pricing charged through the API is **always server-computed** and identical to the dashboard quote for the same inputs.	Parity test: `POST /v1/quotes` == `POST /order/business/route/estimate/` to the cent.
G3	Every API-created aerrand flows through the **existing fulfilment pipeline** unchanged (Django `Order` -> Mongo `deliveries` -> Node driver feed -> Flutter driver app -> RTDB tracking).	No new code paths in the driver backend beyond webhook outbox writes.
G4	Integrators receive **push notifications (webhooks)** for every lifecycle transition, including driver-side transitions that today only exist in Mongo/RTDB.	`aerrand.accepted` … `aerrand.delivered` delivered within 10 s of the driver action (p95).
G5	Keys are **least-privilege, revocable, auditable** and never stored or logged in plaintext.	Security checklist in §9 passes review.
G6	The public surface is **isolated** from the legacy `/order/*` and `/custom-user/*` compatibility endpoints (which are largely `AllowAny` and keyed on unguessable UUIDs).	API keys are rejected on every non-`/v1` route; `/v1` is served only on `api.aerrand.com`.

2.2 Non-goals (v1)

- Aerrand Guard (`order_type="aerrand_guard"`) via API. The vendor confirmation flow is email/human driven (Didit KYC + Interac payout) and is not automatable today.
- OAuth 2.0 / third-party platform apps acting on behalf of many businesses (Shopify-app style). Keys are per-business only.
- Postpaid invoicing or credit terms. v1 payment is **wallet (prepaid)** or **hosted Stripe Checkout**; card-on-file is Phase 2.
- Replacing the business dashboard's own booking flow with `/v1`. The dashboard keeps using `/order/*`; convergence is a follow-up.
- Customer-side (aerrandweb / mobile) developer access.

---

3. Current-state analysis (as built)

Everything in this section is derived from the code as of commit `2075290` on the `staging` branch. It is the baseline the design must respect.

3.1 Topology

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

```
aerrand.com, www.aerrand.com     -> aerrand-website (Next.js)
app.aerrand.com                  -> aerrandweb SPA + /api/webapp/* -> aerrand-api:8000 (prefix stripped)
                                                   /api/customer/* -> aerrand-customer-backend:3000
                                                   /api/driver/*   -> aerrand-driver-backend:3000
business.aerrand.com             -> aerrand_business SPA + /api/webapp/* -> aerrand-api:8000
admin.aerrand.com                -> aerrand_admin SPA    + /api/webapp/* -> aerrand-api:8000
status.aerrand.com               -> uptime-kuma
```

- Single droplet, Docker Compose (`docker-compose.prod.yml`), images pulled from DigitalOcean Container Registry, deployed by GitHub Actions on push to `staging` (`.github/workflows/deploy.yml`).
- One shared MongoDB database used by **all** backends (Django via `django-mongodb-backend`, Node via mongoose). Django `Order` rows are mirrored into the shared `deliveries` collection by `domain.services.DeliveryService.sync_from_order`.
- Django runs under gunicorn with **3 workers** (`aerrand_api/Dockerfile`). There is **no Redis** and no `CACHES` setting (Django default = per-process `LocMemCache`).
- Every container reads the single root `.env`.

3.2 Authentication today

Surface	Mechanism	Notes
Business / admin / customer web dashboards	DRF `TokenAuthentication` — `Authorization: Token <40-hex>` (`REST_FRAMEWORK.DEFAULT_AUTHENTICATION_CLASSES` in `aerrand_api/settings.py`; `aerrand_business/src/app/lib/axios.ts`)	One token per user, issued by `custom_user.views.sign_in` / `sign_up` (`Token.objects.get_or_create`). Stored **plaintext** in Mongo (`authtoken_token`). Never expires, no scopes, no rotation.
Default permission	`AllowAny`	Most `/order/*` views are unauthenticated and rely on unguessable UUIDs. A handful use `IsAuthenticated` (`business_profile`, `business_verification`, `firebase_chat_token`) or the ad-hoc `_require_admin` gate.
Stripe account selection	**Origin header** (`order.views._resolve_frontend_url` -> `_resolve_stripe_key`)	Business dashboard origin -> `STRIPE_SECRET_KEY_BUSINESS`; anything else (including server-to-server calls with no `Origin`) -> customer key.
Business approval	`custom_user.business_approval`	`awaiting_documents -> pending -> approved/rejected`; `block_response()` returns **412** `business_not_approved` on `POST /order/`, `POST /order/batch/checkout/`, `POST /order/<id>/payment-link/`. `null` status = legacy = approved. Team members (`account_type="business_<role>"`) inherit via `CustomUser.invited_by`.

`custom_user.jwt_utils` (HS256 JWT) exists but is only used for the websocket token; it is not the API auth mechanism.

3.3 How a business aerrand is created today

Sequence performed by `aerrand_business/src/app/pages/CreateAerrand.tsx` against `aerrand_api`:

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

```
sequenceDiagram
    participant D as Business dashboard
    participant API as aerrand_api (Django)
    participant M as Mongo deliveries
    participant S as Stripe
    participant N as Node driver backend

    D->>API: POST /order/business/route/estimate/ {pickup, stops[{lat,lng,parcels}]}
    API-->>D: route breakdown (chargeTotal incl. HST)
    loop per stop (1..5)
        D->>API: POST /order/ {business:true, price, size, vehicle_type, date_time, batch_*,
          pickup_location, dropoff_location}
        API->>API: business_approval.block_response()  (412 if not approved)
        API->>API: Order(status="draft", channel="business") — client price TRUSTED
        API->>M: DeliveryService.sync_from_order -> status pending_payment
        API-->>D: 201 {data.id, payment_link}
        D->>API: PUT /order/{id}/edit-pickup-instruction/
        D->>API: PUT /order/{id}/edit-dropoff-instruction/
    end
    D->>API: POST /order/batch/checkout/ {order_ids}   (multi-stop)  — or per-order link
    API->>S: checkout.Session.create(metadata.order_ids)
    S-->>D: hosted checkout (human pays)
    S->>API: webhook checkout.session.completed
    API->>API: _mark_order_paid_by_id -> status pending -> _start_driver_search -> broadcasted
    API->>M: sync_from_order + _confirm_delivery_paid
    API->>N: (via Mongo) delivery visible in driver feed
    N->>M: accept / arrive / pickup / deliver — writes the delivery doc directly
    D->>API: GET /order/{id}/tracking/ (polls live delivery doc)
```

Key facts:

- **Pricing**: for `business: true` orders the server **trusts the client** `price` (`order.views.order_list`, comment "That client price already comes from an authoritative server endpoint, so we trust it"). The customer flow recomputes; the business flow does not. Same for `total_cargo_units` (van-capacity cap).
- **Batch**: multi-stop = N `Order` rows sharing `batch_id/batch_index/batch_size`, mirrored as `batch{}` on the delivery doc; the Node backend groups them into one driver job.
- **Status machine**: `draft -> pending -> broadcasted -> accepted -> arrived -> picked_up -> in_transit -> delivered | cancelled` (canonical map in `domain/mappers.py`). Unpaid + unassigned orders are demoted to `pending_payment` on the delivery doc so the driver feed (`DRIVER_VISIBLE_OFFER_STATUSES = ['upcoming','pending','broadcasted','draft']`) never shows them.
- **Payment**: Stripe Checkout only. Wallet (`custom_user.models.Wallet`, `WalletTopup`) exists for businesses and is funded by Stripe, but **nothing debits it** for orders.
- **Driver-side transitions** happen in Node (`aerrand_driver_backend/controllers/deliveryController.js`) directly on the Mongo doc and RTDB; Django only sees them when a client polls `/order/{id}/tracking/` (`_fetch_live_delivery_doc`).
- **Notifications**: `notification.services.notify_order_status` writes a durable `Notification` row + RTDB `user_notifications/customer_{id}` for Django-originated transitions only.
- **Cancellation**: `POST /order/{id}/cancel/` resolves a fee from `lifecycle_stage` via `business_cancellation`; Stripe reconciliation is explicitly "wired in a later phase" (fee is recorded, not charged/refunded).

3.4 Gaps that block simply reusing the dashboard flow with an API key

#	Gap	Consequence for an API channel	Resolution in this spec
GAP-1	Client-supplied `price` / `total_cargo_units` are trusted for business orders.	A key holder could book at $0.50.	`/v1` computes price server-side from `stops[].parcels` + routed km (FR-CREATE-04). Legacy path untouched.
GAP-2	Stripe key chosen by `Origin`. Server-to-server calls have no `Origin` -> customer Stripe account.	Business API payments would land in the wrong Stripe account.	`/v1` resolves the Stripe key from the **key owner's channel**, never from `Origin` (FR-PAY-06).

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

#	Gap	Consequence for an API channel	Resolution in this spec
GAP-3	Payment requires a human on a hosted checkout page.	No fully automated booking.	Wallet debit at creation as the default payment mode (FR-PAY-01..05). Checkout mode kept for human-in-the-loop.
GAP-4	Driver transitions are invisible to Django until polled.	Cannot emit `aerrand.accepted/…/delivered` webhooks reliably.	Shared Mongo `webhook_outbox` written by both Django and Node, drained by a dispatcher service (§4.6, FR-WH-*).
GAP-5	DRF tokens are plaintext, unscoped, non-expiring, per-person.	Not acceptable as developer credentials.	New `ApiKey` model: hashed, scoped, revocable, business-owned (§5.1, §7).
GAP-6	No rate limiting, no idempotency, 3 gunicorn workers with per-process cache.	Retry storms double-book; abuse unthrottled.	Redis-backed throttles + `Idempotency-Key` (FR-RATE-, *FR-IDEM-*).
GAP-7	Order creation is 1 + 2 calls per stop (create, then pickup/dropoff instruction edits).	Non-atomic; partial routes possible.	Single atomic `POST /v1/aerrands` with inline contacts (`AddOrderSerializer` already accepts nested `pickup_instruction`/`dropoff_instruction`).
GAP-8	`Order.channel` is `business` or `customer`; dashboard filters `channel=business`.	API aerrands would be invisible in the dashboard (or indistinguishable).	New channel value `api` + dashboard filter widened to `business,api` with an "API" badge (FR-CREATE-09, FR-UI-07).

4. Target architecture

4.1 Overview

```
flowchart LR
    subgraph Integrator
        C[Business system<br/>ERP / storefront]
        R[Webhook receiver]
    end
    subgraph Edge
        CD[Caddy<br/>api.aerrand.com]
    end
    subgraph Django aerrand_api
        V1[/v1 views<br/>ApiKeyAuthentication + scopes/]
        SVC[developer.services<br/>aerrands · wallet · webhooks]
        LEG[order.* legacy views<br/>unchanged]
        DISP[webhook dispatcher<br/>manage.py run_webhook_dispatcher]
    end
    subgraph Data
        MG[(MongoDB<br/>orders · deliveries · api_keys · webhook_outbox …)]
        RD[(Redis<br/>throttles)]
    end
    N[Node driver backend]
    F[Flutter driver app]
    ST[Stripe]

    C -- Bearer aer_live_… --> CD --> V1 --> SVC
    SVC --> MG
    V1 --> RD
    SVC -. reuse .-> LEG
    SVC --> ST
    N --> MG
    F --> N
    N -- outbox insert --> MG
    SVC -- outbox insert --> MG
    DISP -- poll/claim --> MG
    DISP -- signed POST --> R
```


+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

4.2 Public host and routing

- New public host `api.aerrand.com` (DNS A record -> droplet; Caddy provisions TLS).
- Caddy proxies **only** `/v1/*`, `/v1/openapi.json`, `/docs`, and `/health` to `aerrand-api:8000`. Everything else on this host returns 404. The legacy Django routes (`/order/*`, `/custom-user/*`, `/admin/`, `/swagger/`) are **not** reachable on this host.
- `/v1/*` is **not** served on `app.`, `business.` or `admin.aerrand.com`. This keeps the public API on a clean origin with no CORS, no cookies, no CSRF and no `Origin`-based behaviour.
- Django mounts `path('v1/', include('developer.urls_v1'))` in `aerrand_api/urls.py`.

4.3 New Django app: `developer`

```
aerrand_api/developer/
  models.py            ApiKey, ApiQuote, IdempotencyRecord, WebhookEndpoint, WebhookOutbox,
    WebhookDelivery, ApiRequestLog
  authentication.py    ApiKeyAuthentication (DRF BaseAuthentication)
  permissions.py       HasApiScope("aerrands:write") etc.
  throttling.py        PerKeyThrottle, CreateThrottle, QuoteThrottle
  idempotency.py       @idempotent view decorator
  serializers.py       v1 request/response schemas
  services/
    keys.py            generate / hash / verify / rotate / revoke
    quotes.py          route_km + business_pricing.route_price, ApiQuote persistence
    aerrands.py        create_aerrand(), get_aerrand(), list_aerrands(), cancel_aerrand()
    wallet.py          debit_for_aerrand(), refund_for_cancellation()
    webhooks.py        enqueue(), sign(), deliver()
    public_shape.py    Order/Delivery -> public Aerrand object
  views_v1.py          /v1/* endpoints
  views_dashboard.py   /custom-user/api-keys/*, /custom-user/webhooks/* (Token-auth, dashboard)
  views_admin.py       /order/admin/businesses/<id>/api-keys/* (admin)
  management/commands/run_webhook_dispatcher.py
  management/commands/expire_unpaid_api_aerrands.py
  migrations/
  tests/
```

4.4 Request pipeline (`/v1/*`)

1. Caddy terminates TLS, forwards `X-Forwarded-Proto`, injects `X-Request-Id` if absent.
2. `ApiKeyAuthentication` parses `Authorization: Bearer aer_(live|test)_…`, hashes it, looks up `api_keys.key_hash` (unique index), rejects revoked/expired keys, resolves `request.user = business owner CustomUser`, `request.auth = ApiKey`. Sets `request.api_environment`.
3. `HasApiScope` checks the view's required scope against `api_key.scopes`.
4. Business eligibility: `business_approval.block_response(owner)` — 412 on non-approved (identical rule to the dashboard).
5. Throttles (Redis) — 429 with `Retry-After`.
6. `@idempotent` (mutating endpoints only) — replays stored response or claims the key.
7. Service layer executes; every DB read is scoped to `customer_id == owner.id`.
8. Response: JSON, `X-Request-Id`, `X-RateLimit-*` headers. `ApiRequestLog` row written asynchronously (fail-safe).

4.5 Payment models

Mode	v1?	Flow	When to use
`wallet` (default)	Yes	Debit `Wallet.balance` atomically at create; order is paid immediately and broadcast to drivers. Refund-to-wallet on cancellation minus policy fee.	Fully automated integrations. Business tops up via dashboard (`wallet_topup_checkout`) or auto-refill (Phase 2).

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

Mode	v1?	Flow	When to use
`checkout`	Yes	Create Stripe Checkout (business key) and return `checkout_url`; aerrand stays `pending_payment` until Stripe webhook. Auto-cancelled after `API_UNPAID_TTL_HOURS`.	Human-in-the-loop (agent approves and pays).
`card_on_file`	Phase 2	Stripe Customer + SetupIntent captured in the dashboard; off-session PaymentIntent at create; `requires_action` fallback.	Automated integrations that don't want prepaid balance.
`invoice`	Phase 3	Postpaid, monthly statement.	Enterprise accounts.

4.6 Capturing driver-side transitions for webhooks (outbox pattern)

Django cannot observe Node's writes to `deliveries`. Options considered:

Option	Verdict
Poll `deliveries` from Django for `channel="api"` docs	Simple but O(active orders) every few seconds, adds latency, misses transient states. Rejected.
Mongo Change Streams on `deliveries`	Needs a long-running consumer anyway; brittle across the `_id`/`id` dual-key quirk and BSON-date-in-JSONField issue noted in `order_tracking`. Rejected for v1.
RTDB listener on `user_notifications`	Firebase rules/uid fan-out are already fragile (see business realtime notes). Rejected.
Transactional outbox — both backends insert an event row into `webhook_outbox` at the exact places they already push RTDB notifications; a dispatcher drains it	Explicit, testable, no new infra beyond a worker process. **Chosen.**

Writers:

- Django: `notification.services.notify_order_status` (all status transitions it sees), `_mark_order_paid_by_id` (`aerrand.paid`), `cancel_order` / `cancel_aerrand` (`aerrand.cancelled`), `create_aerrand` (`aerrand.created`).
- Node driver backend: a new `services/webhookOutbox.js` called from `deliveryController.js` at accept / arrived / picked_up / in_transit / delivered / driver-cancel, guarded by `delivery.channel === 'api'` (or the owner having >=1 enabled endpoint — the dispatcher drops events with no subscriber anyway).

Dispatcher: `aerrand-webhook-dispatcher` compose service = the `aerrand-api` image running `python manage.py run_webhook_dispatcher`. Polls every `WEBHOOK_DISPATCHER_POLL_SECONDS` (2 s), claims rows with `findOneAndUpdate` lease (`claimed_at`, `claimed_by`, 60 s expiry), builds the public Aerrand snapshot, signs and POSTs, records `webhook_deliveries`, schedules retries. Single replica in v1 (lease makes it safe to scale later).

---

5. Functional requirements

Requirement IDs are stable; priority uses MoSCoW (**M**ust / **S**hould / **C**ould).

5.1 API key lifecycle

ID	Pri	Requirement
FR-KEY-01	M	A key is owned by a **business owner account** (`CustomUser` with `BusinessProfile`), never by an individual team member. `ApiKey.business_owner_id` = `business_approval.resolve_business_owner(creator).id`.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

ID	Pri	Requirement
FR-KEY-02	M	Only the owner (`account_type="business"`) and members with role `business_admin` may create, rotate or revoke keys. Other roles (`business_dispatcher`, `business_manager`, viewer) may list keys (prefix only).
FR-KEY-03	M	Keys can be created only when `business_approval.effective_status(owner) == "approved"`. A non-approved business sees the Developers section disabled with the same copy as `BLOCK_MESSAGES`.
FR-KEY-04	M	Key format: `aer_<env>_<40 chars base62>` where `env in {live, test}`; entropy >= 230 bits (`secrets.token_bytes(32)` base62-encoded). Example: `aer_live_7Kc2QmXv9pT4RbLw8ZsE1nHyU3aDfGjM0oPqIiVx`.
FR-KEY-05	M	The full key is returned **exactly once** (creation and rotation responses) and never again. Persisted fields: `key_prefix` (`aer_live_7Kc2Qm`, first 6 random chars, for display/search) and `key_hash = HMAC-SHA256(API_KEY_HASH_PEPPER, key)`. The plaintext key is never written to Mongo, logs, Sentry or request logs.
FR-KEY-06	M	Attributes at creation: `name` (1–64 chars, unique per business), `environment`, `scopes` (subset of §5.2 list, default all non-admin scopes), optional `expires_at` (<= 1 year), optional `ip_allowlist` (<= 20 CIDRs).
FR-KEY-07	M	**Revoke** is immediate and irreversible (`status="revoked"`, `revoked_at`, `revoked_by`). Requests with a revoked key get `401 api_key_revoked`. No caching of key validity longer than the request (v1 does one indexed Mongo read per request).
FR-KEY-08	S	**Rotate**: creates a new key with the same name/scopes/environment and puts the old key into a 24-hour grace period (`status="rotating"`, `expires_at = now + 24h`), after which it is revoked automatically. Grace period configurable 0–72 h per rotation call.
FR-KEY-09	M	Limits: <= 10 active keys per environment per business (admin override).
FR-KEY-10	M	`last_used_at` and `last_used_ip` are updated at most once per 60 s per key (write-coalesced) so authentication does not add a write per request.
FR-KEY-11	M	Every create/rotate/revoke writes an audit row (`api_audit_log`) with actor, key id, action, IP, user agent, and sends an email to the owner (`ray_send_email`, Resend) — "A new API key `aer_live_7Kc2Qm…` was created by `<actor email>`".
FR-KEY-12	S	Global kill switch `API_KEYS_ENABLED=false` -> every `/v1` request returns `503 api_disabled`. Per-business switch `BusinessProfile.api_access_enabled` (default `true` once approved) toggled by admin -> `403 api_access_disabled`.
FR-KEY-13	C	Owner-initiated "revoke all keys" panic button in the dashboard.

5.2 Authentication and authorization

ID	Pri	Requirement
FR-AUTH-01	M	Header: `Authorization: Bearer <key>`. `X-Api-Key: <key>` is accepted as an alias. Query-string keys are rejected (`400 api_key_in_query`).
FR-AUTH-02	M	`ApiKeyAuthentication` is attached **explicitly** to `/v1` views (`authentication_classes = [ApiKeyAuthentication]`). The global `REST_FRAMEWORK` defaults are unchanged; DRF `Token` auth is **not** accepted on `/v1`, and API keys are **not** accepted on any legacy route.
FR-AUTH-03	M	Scopes: `quotes:read`, `aerrands:read`, `aerrands:write`, `aerrands:cancel`, `wallet:read`, `webhooks:manage`. Each endpoint declares exactly one required scope (§6.4). Missing scope -> `403 insufficient_scope` with `required_scope` in the body.
FR-AUTH-04	M	Environment binding: a `test` key can only read/write `test_mode=true` aerrands and vice versa. Cross-environment reads return 404 (not 403) to avoid existence leaks.
FR-AUTH-05	M	IP allowlist, when set, is enforced against the client IP from `X-Forwarded-For` (first hop, Caddy is the only trusted proxy). Mismatch -> `403 ip_not_allowed`.
FR-AUTH-06	M	Failed authentication responses are uniform (`401 invalid_api_key`) and constant-time with respect to whether the prefix exists (lookup is by hash; no early return on prefix).
FR-AUTH-07	M	All `/v1` responses include `Cache-Control: no-store`.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

5.3 Business eligibility

ID	Pri	Requirement
FR-ELIG-01	M	Every mutating `/v1` request re-evaluates `business_approval.block_response(owner)`; a rejected/suspended business gets `412 business_not_approved` with the same body shape the dashboard receives. Reads remain allowed so an integrator can see historical aerrands.
FR-ELIG-02	M	Approval status changes (admin approve/reject) do not touch keys, but a rejection blocks creation from the next request onward.
FR-ELIG-03	S	`GET /v1/me` exposes `approval_status`, `can_create_aerrands`, `api_access_enabled` and wallet summary so integrators can self-diagnose 412s.

5.4 Quotes

ID	Pri	Requirement
FR-QUOTE-01	M	`POST /v1/quotes` accepts `pickup {lat,lng}` and `stops[1..5] {lat,lng,parcels{small,medium,large,xl}}` and returns the full `business_pricing.route_price()` breakdown plus `recommended_vehicle`, `route_km`, `total` (= `chargeTotal`, HST-inclusive) — the same numbers as `POST /order/business/route/estimate/`.
FR-QUOTE-02	M	Route km = sum of sequential legs pickup->stop1->…->stopN via `order.views.resolve_distance_and_drive_min` (Google Distance Matrix, haversine ×1.3 fallback). Stops are priced in the order given; v1 does **not** re-optimise stop order (matches the dashboard).
FR-QUOTE-03	M	Each quote is persisted (`api_quotes`) with `quote_id = qt_<uuid>`, inputs hash, breakdown and `expires_at = now + API_QUOTE_TTL_SECONDS` (default 600).
FR-QUOTE-04	M	Validation errors: > 5 stops -> `400 too_many_stops`; total cargo units > 20 (`van_capacity_cap`) -> `400 over_van_capacity`; no parcels on a stop -> `400 stop_without_parcels`; coordinates outside `API_SERVICE_AREA_BBOX` (when configured) -> `400 outside_service_area`.
FR-QUOTE-05	S	Quotes may include optional address strings; when `lat/lng` are omitted and `address` is present, the server geocodes with Google (`GOOGLE_MAPS_API_KEY`) and returns the resolved coordinates. Geocoding failures -> `422 address_not_found`.

5.5 Aerrand creation

ID	Pri	Requirement
FR-CREATE-01	M	`POST /v1/aerrands` creates a single-stop or multi-stop (<= 5) aerrand in **one call**, including pickup contact, per-stop recipient, instructions, parcels, optional `scheduled_at`, `vehicle`, `external_reference`, `metadata`, and `payment {mode}`. Request schema in §6.4.
FR-CREATE-02	M	Creation is **atomic** across stops: either all `Order` rows (+ `PickupLocation`, `DropOffLocation`, `PickupInstruction`, `DropOffInstruction`) exist and are paid/pending together, or none do. Implementation: `transaction.atomic()` where `django-mongodb-backend` supports it on the deployed replica set; otherwise explicit compensation (delete created rows) on any failure before the success response.
FR-CREATE-03	M	Each stop becomes one `Order` row (`order_type="single"` for N=1, `"multi"` for N>1, matching the dashboard's `deliveryType` values). For N>1 the rows share `batch_id` (server-minted UUID), `batch_index` (0-based, request order) and `batch_size=N`. For N=1 batch fields stay `null` (identical to dashboard semantics, so nothing downstream changes).
FR-CREATE-04	M	**Price is computed server-side**: if `quote_id` is supplied and unexpired and its inputs hash matches the request's pickup/stops, the stored breakdown is reused; otherwise the route is re-priced. The client never sends `price`. Optional `max_total` -> `409 price_exceeds_max` when the computed total is higher (returns the computed quote).
FR-CREATE-05	M	Per-stop `Order.price` = that stop's share of `chargeTotal` (equal split, cents reconciled on the last stop so the sum equals the total exactly — the dashboard uses `total / N`). `delivery_fee`, `hst`, `price_breakdown` (full route breakdown + `stopShare`) are populated on every row so admin reports work.
FR-CREATE-06	M	`Order.size` is set to the lbs-equivalent of the largest parcel class on the stop (`small->"5"`, `medium->"15"`, `large->"30"`, `xl->"60"`, the dashboard's `SIZE_TO_INT`) so the driver app displays the same size tier. `Order.volume` = total parcel count on the stop.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

ID	Pri	Requirement
FR-CREATE-07	M	`vehicle` is optional; default = `recommended_vehicle` from pricing. A supplied vehicle must be >= the recommended tier (`scooter < motorcycle < car < truck`), else `400 vehicle_below_requirement`. Persisted as `Order.vehicle_type`.
FR-CREATE-08	M	`scheduled_at` (ISO-8601, optional): must be >= now - 5 min and <= now + 7 days; persisted as `Order.date_time`. Absent = ASAP. (Dispatch timing for scheduled orders is unchanged from today — see OQ-6.)
FR-CREATE-09	M	`Order.customer_id = owner.id` (server-set), `Order.channel = "api"`, `Order.api_key_id`, `Order.external_reference` (<= 128 chars, unique per business per environment — duplicate -> `409 duplicate_external_reference` unless `allow_duplicate_reference=true`), `Order.metadata` (<= 20 keys, keys <= 40 chars, string values <= 500 chars), `Order.test_mode`, `Order.payment_mode`.
FR-CREATE-10	M	Pickup contact defaults to the business profile (`BusinessProfile.contact_name`, `phone`, `contact_email`) when omitted. Recipient `name` and `phone` are required per stop (E.164 validated). `dropoff_preference in {leave_at_door, hand_to_recipient, reception, other}` maps to `DropOffInstruction.pick_up_prefrence`.
FR-CREATE-11	M	After rows exist, `DeliveryService.sync_from_order(order, endpoint="v1_create")` runs per row (skipped entirely when `test_mode`). Unpaid rows sync as `pending_payment`; paid rows sync as `broadcasted` — exactly the current mapper behaviour.
FR-CREATE-12	M	Response is `201` with the full public Aerrand object (§6.3) and, in `checkout` mode, `payment.checkout_url`.
FR-CREATE-13	M	Guard/escrow inputs (`declared_item_value`, `order_type="aerrand_guard"`) are rejected in v1 (`400 unsupported_order_type`).
FR-CREATE-14	S	`notify_order_status`-style in-app notification + RTDB push to the owner (and linked team members) on creation so the dashboard's live feed shows API aerrands without a refresh.

5.6 Payment

ID	Pri	Requirement
FR-PAY-01	M	`payment.mode` in `{wallet, checkout}`; default `wallet`. Persisted as `Order.payment_mode`.
FR-PAY-02	M	**Wallet debit is atomic and race-safe**: `Wallet.objects.filter(user=owner, balance__gte=total).update(balance=F("balance") - total)`; `0` rows updated -> `402 insufficient_funds` with `{required, available, currency, topup_url}` and **no rows are left behind**. Amounts use `Decimal` quantised to cents.
FR-PAY-03	M	A successful debit writes a `WalletTransaction(transaction_type="aerrand_charge", amount=-total, order_id=<batch_id or order id>, description=…)`. `WalletTransaction.TRANSACTION_TYPES` gains `aerrand_charge` and `aerrand_refund`; a nullable `order_id` field is added.
FR-PAY-04	M	After debit, every row goes through the existing paid path (`_mark_order_paid_by_id` refactored to accept an order + reference): `payment_status=True`, `payment_session_id="wallet:<txn_id>"`, status `pending -> broadcasted` via `_start_driver_search`, `sync_from_order`, `_confirm_delivery_paid`, `realtime.sync_order_status`, `notify_order_status(order,"broadcasted")`, outbox `aerrand.paid`.
FR-PAY-05	M	Wallet mode in `test` environment performs no debit and creates no transaction; the response still reports `payment.status="paid"` with `reference="test"`.
FR-PAY-06	M	`checkout` mode calls `create_payment_link()` with the Stripe key resolved **explicitly for the business channel** (`_resolve_stripe_key(settings.AERRAND_BUSINESS_WEB_URL)` — never from `Origin`), metadata `{order_ids, type:"batch_order", source:"api", aerrand_id}` (single-stop uses `order_id`), and optional client `success_url` / `cancel_url` (https only) defaulting to the business dashboard pages. The existing `stripe_webhook` marks the rows paid unchanged.
FR-PAY-07	M	Unpaid `checkout` aerrands are auto-cancelled (no fee) by `expire_unpaid_api_aerrands` after `API_UNPAID_TTL_HOURS` (default 24) and emit `aerrand.cancelled` with `reason="payment_expired"`. The Stripe session is expired via `stripe.checkout.Session.expire`.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

ID	Pri	Requirement
FR-PAY-08	M	Prerequisite flagged: `STRIPE_SECRET_KEY_BUSINESS` is currently a **test-mode** key (per `settings.py` comments). Live API checkout requires the business Stripe account to be switched to live keys and a matching `STRIPE_WEBHOOK_SECRET_BUSINESS`.
FR-PAY-09	S	`GET /v1/wallet` returns `balance`, `currency`, `pending_topups`, and the last 20 transactions; `topup_url` deep-links to the dashboard Wallet section.
FR-PAY-10	C	Low-balance webhook `wallet.balance_low` when balance drops below a business-configured threshold.

5.7 Reads, listing and tracking

ID	Pri	Requirement
FR-READ-01	M	`GET /v1/aerrands/{id}` where `{id}` is the batch id (multi-stop) or the order id (single-stop); the server resolves either, scoped to the owner. Live fields (driver, driver location, stage timestamps, per-stop status) are merged from the `deliveries` doc via the existing `_fetch_live_delivery_docs_by_ids` / `_tracking_payload_from_doc` helpers.
FR-READ-02	M	`GET /v1/aerrands` supports `status`, `external_reference`, `created_after`, `created_before`, `scheduled_after`, `scheduled_before`, `limit` (<= 100, default 20) and opaque cursor pagination (`starting_after`). Sorted by `created_at desc`. Multi-stop siblings are collapsed into one item.
FR-READ-03	M	`GET /v1/aerrands/{id}/tracking` returns the lightweight live view (status, driver, `driver_location{lat,lng,heading,speed,updated_at}`, ETA when available, per-stop timeline) and the public `tracking_url` (`{AERRAND_BUSINESS_WEB_URL}/track/{order_id}` — the existing token-less tracking page backed by `public_order_track`).
FR-READ-04	S	`GET /v1/aerrands/{id}/proof` returns delivery photos / handoff data for delivered stops (`photos` from the delivery doc), as short-lived signed URLs.
FR-READ-05	M	`GET /v1/events` lists the last 30 days of webhook events for the business (for reconciliation), filterable by `type`, `aerrand_id`, cursor-paginated.

5.8 Cancellation

ID	Pri	Requirement
FR-CANCEL-01	M	`POST /v1/aerrands/{id}/cancel` (scope `aerrands:cancel`) cancels **all non-terminal stops** of the aerrand using the existing fee engine (`business_cancellation.resolve_business_cancellation` per row with server-derived `wait_minutes` / `driver_distance_km`, exactly as `order.views.cancel_order`). Refactor the body of `cancel_order` into `order.services.cancel_business_order(order, initiated_by)` used by both.
FR-CANCEL-02	M	If any stop is at a stage where cancellation is not allowed (`CancellationNotAllowed`) the whole call returns `409 cancellation_not_allowed` with per-stop reasons and **nothing is cancelled** (all-or-nothing), unless `partial=true` is passed.
FR-CANCEL-03	M	Wallet-paid aerrands: refund = `paid_total - Sum fee` credited back atomically (`F("balance") + refund`) with a `WalletTransaction(aerrand_refund)`. Response includes `{fee, refund, aerrander_compensation, from_stage}`. `checkout`-paid aerrands record the fee only (Stripe reconciliation remains the existing "later phase" — OQ-4).
FR-CANCEL-04	M	Emits `aerrand.cancelled` with `reason in {business_request, payment_expired, driver_no_show, admin}` and the fee outcome.
FR-CANCEL-05	C	`PATCH /v1/aerrands/{id}` for pre-dispatch edits (instructions, recipient phone) using `edit_*_instruction` internals; address/parcel changes require cancel + recreate in v1.

5.9 Webhooks

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

ID	Pri	Requirement
FR-WH-01	M	A business can register <= 5 endpoints (`POST /v1/webhooks` or dashboard): `url` (https only, <= 2048 chars), `events` (list or `["*"]`), `description`. Response returns the signing secret **once**: `aer_whsec_<40 base62>`.
FR-WH-02	M	The secret is stored **encrypted at rest** (Fernet, `WEBHOOK_SECRET_ENCRYPTION_KEY`, `cryptography` added to `requirements.txt`) because the dispatcher must read it to sign. Never returned after creation; `POST /v1/webhooks/{id}/rotate-secret` issues a new one with a 24 h overlap (both secrets sign during overlap: `v1=` and `v1=` twice in the header, Stripe-style).
FR-WH-03	M	Event catalogue (§6.5): `aerrand.created`, `aerrand.paid`, `aerrand.searching`, `aerrand.accepted`, `aerrand.arrived_at_pickup`, `aerrand.picked_up`, `aerrand.in_transit`, `aerrand.stop_delivered`, `aerrand.delivered`, `aerrand.cancelled`, `aerrand.payment_expired`, `wallet.balance_low` (C).
FR-WH-04	M	Envelope: `{ "id": "evt_<uuid>", "type", "api_version": "2026-08-29", "created": <unix>, "test_mode", "data": { "object": <Aerrand> , "previous_status" } }`. Snapshot is built at dispatch time (latest state) but `type`/`previous_status` reflect the transition that enqueued it.
FR-WH-05	M	Signature: `Aerrand-Signature: t=<unix seconds>,v1=<hex HMAC-SHA256(secret, "<t>.<raw body>")>`. Receivers must reject `
FR-WH-06	M	Delivery semantics: at-least-once, ordered per aerrand on a best-effort basis (dispatcher processes an aerrand's events FIFO); receivers must dedupe on `id`. Success = any `2xx` within `WEBHOOK_TIMEOUT_SECONDS` (10). No redirects followed.
FR-WH-07	M	Retry schedule on failure: 1 m, 5 m, 30 m, 2 h, 12 h, 24 h (6 attempts, `WEBHOOK_MAX_ATTEMPTS`). After the last failure the event is marked `failed`; after 7 consecutive days of failures the endpoint is auto-disabled (`disabled_reason="unreachable"`) and the owner is emailed.
FR-WH-08	M	SSRF protection: on create and again at send time, resolve the hostname and refuse loopback, RFC1918, link-local, CGNAT, multicast and metadata ranges; refuse non-443/80 ports unless explicitly allowed; refuse `.local`/`.internal`.
FR-WH-09	M	`POST /v1/webhooks/{id}/test` sends a synthetic `aerrand.created` with `test_mode=true`; `POST /v1/events/{id}/redeliver` re-enqueues a stored event to all matching endpoints. Delivery attempts (status, latency, response snippet <= 1 KB) are visible via `GET /v1/webhooks/{id}/deliveries` and the dashboard.
FR-WH-10	M	Outbox writers (Django + Node) insert `{event_type, order_id, batch_id, business_owner_id, channel, previous_status, created_at, status:"pending"}`; the dispatcher builds the payload — writers never serialise the full object (keeps Node changes tiny and the public shape in one place).
FR-WH-11	M	Events are only enqueued for orders with `channel="api"` **or** whose owner has >= 1 enabled endpoint (so dashboard-created aerrands can also be observed by integrators).

5.10 Idempotency

ID	Pri	Requirement
FR-IDEM-01	M	`POST /v1/aerrands`, `POST /v1/aerrands/{id}/cancel`, `POST /v1/webhooks` accept an `Idempotency-Key` header (1–255 chars). Strongly recommended in docs; required for `POST /v1/aerrands` in `live` (`400 idempotency_key_required`).
FR-IDEM-02	M	Scope = `(api_key_id, key)`. First request claims the key (`status="in_progress"`, unique index) and stores `fingerprint = SHA-256(method + path + canonical body)`. Completion stores `response_status` + `response_body`. TTL `API_IDEMPOTENCY_TTL_HOURS` (24) via Mongo TTL index.
FR-IDEM-03	M	Replay with the same fingerprint -> the stored response (same status code, body, plus `Idempotent-Replayed: true`). Same key, different fingerprint -> `422 idempotency_key_reused`. Key still `in_progress` (concurrent) -> `409 idempotency_in_progress` with `Retry-After: 2`.
FR-IDEM-04	M	If the original request failed with 5xx, the claim is released so the client can retry.

5.11 Rate limiting and quotas

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

ID	Pri	Requirement
FR-RATE-01	M	Per-key sliding-window limits (defaults, overridable per business by admin): all endpoints 60 req/min and 3,000 req/hour; `POST /v1/aerrands` 20/min; `POST /v1/quotes` 120/min; `test` keys ×2.
FR-RATE-02	M	Storage: Redis (`redis:7-alpine` compose service, `django-redis`, `REDIS_URL`). If `REDIS_URL` is unset the throttles fall back to `LocMemCache` with a startup warning — acceptable for local dev only (3 gunicorn workers make per-process limits ~ 3× looser).
FR-RATE-03	M	Headers on every response: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` (unix). Exceeded -> `429 rate_limited` with `Retry-After`.
FR-RATE-04	S	Daily creation quota per business (default 2,000 aerrands/day) -> `429 daily_quota_exceeded`; admin-adjustable.
FR-RATE-05	M	Unauthenticated / failed-auth requests are limited per IP (30/min) to blunt key guessing.

5.12 Test mode

ID	Pri	Requirement
FR-TEST-01	M	`aer_test_` keys create aerrands with `Order.test_mode=True`. Test aerrands: are never synced to `deliveries` (no driver visibility), never debit the wallet, never create Stripe sessions, never push RTDB notifications, and are excluded from admin dashboard aggregates and business reports (`admin_views._dashboard_order_aggregates` and report queries add `test_mode != true`).
FR-TEST-02	M	Test aerrands are visible in the business dashboard only behind a "Show test data" toggle (badge "TEST").
FR-TEST-03	S	`POST /v1/test/aerrands/{id}/advance {to: "<public status>"}` (test keys only) walks the aerrand forward through valid transitions, stamping a synthetic driver and timeline, and emits the corresponding webhooks — so integrators can exercise their receiver end-to-end without a driver.
FR-TEST-04	S	Test data older than 30 days is purged by a management command.

5.13 Business dashboard UI (`aerrand_business`)

ID	Pri	Requirement
FR-UI-01	M	New `Profile` section **"Developers"** (`Section = "developers"`, icon `Code`), visible to all roles, actionable per FR-KEY-02. Disabled state with approval copy per FR-KEY-03.
FR-UI-02	M	Keys table: name, prefix (`aer_live_7Kc2Qm…`), environment chip, scopes, created, last used, status; actions Rotate / Revoke (confirm dialog naming the key).
FR-UI-03	M	"Create key" modal: name, environment, scope checkboxes (all on by default), optional expiry; success state shows the full key **once** with copy button and the warning "You won't see this again".
FR-UI-04	M	Webhooks tab: endpoint list (url, events, status, last delivery), add/edit/disable, "Send test event", secret shown once, rotate secret; delivery log per endpoint with status codes, latency and "Redeliver".
FR-UI-05	S	Logs tab: last 7 days of API requests (time, method, path, status, latency, request id) from `api_request_logs`.
FR-UI-06	M	Links to `https://api.aerrand.com/docs` and the quickstart.
FR-UI-07	M	Aerrands lists (`orderApi.byCustomer`) request `channel=business,api` (server accepts a comma list) and render an **"API"** badge + `external_reference` on API-created rows. `DeliveryDetail` shows `external_reference` and `metadata`.
FR-UI-08	M	Dashboard endpoints (Token-authenticated, `IsAuthenticated`): `GET/POST /custom-user/api-keys/`, `POST /custom-user/api-keys/<id>/rotate/`, `DELETE /custom-user/api-keys/<id>/`, `GET/POST /custom-user/webhooks/`, `PATCH/DELETE /custom-user/webhooks/<id>/`, `POST /custom-user/webhooks/<id>/test/`, `GET /custom-user/webhooks/<id>/deliveries/`, `GET /custom-user/api-logs/`. All resolve the business via `resolve_business_owner(request.user)` and enforce role per FR-KEY-02.

5.14 Admin dashboard (`aerrand_admin`)

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

ID	Pri	Requirement
FR-ADMIN-01	M	`BusinessesPage` detail gains an **"API access"** panel: `api_access_enabled` toggle, keys list (prefix, env, created, last used, status) with admin Revoke, rate-limit override, daily quota override.
FR-ADMIN-02	M	Endpoints follow the existing `_require_admin` pattern: `GET /order/admin/businesses/<id>/api-keys/`, `POST …/api-keys/<key_id>/revoke/`, `PATCH /order/admin/businesses/<id>/api-access/`.
FR-ADMIN-03	S	Admin "API overview" card on `DashboardPage`: requests/24 h, aerrands via API/24 h, webhook failure rate, top businesses.
FR-ADMIN-04	S	`OrdersPage` / `DeliveriesPage` filter by `channel=api` and show `external_reference`.

5.15 Developer documentation

ID	Pri	Requirement
FR-DOCS-01	M	Hand-maintained OpenAPI 3.1 spec at `aerrand_api/docs/openapi-v1.json`, served at `GET https://api.aerrand.com/v1/openapi.json`, with Redoc at `https://api.aerrand.com/docs` (same convention as the Node services' `docs/openapi.json` + `/api-docs`).
FR-DOCS-02	M	Quickstart (create key -> quote -> create -> webhook), authentication, idempotency, rate limits, webhook verification samples (Node, Python, PHP), error catalogue, status lifecycle, changelog.
FR-DOCS-03	S	Postman collection generated by extending `scripts/generate-postman-collections.mjs` to the v1 spec.
FR-DOCS-04	C	Minimal official SDKs (TypeScript, Python) generated from the spec.

6. Public API specification (v1)

6.1 Conventions

- Base URL: `https://api.aerrand.com/v1`. JSON request/response, UTF-8. Timestamps ISO-8601 UTC (`2026-08-29T14:03:00Z`). Money as decimal strings in CAD (`"24.85"`), never floats.
- Versioning: path (`/v1`). Backwards-compatible additions (new fields, new event types) are not versioned; breaking changes ship as `/v2`. Webhook envelopes carry `api_version`.
- Request IDs: `X-Request-Id` echoed (or generated). Include it in support requests.
- Max body 256 KB. Unknown fields are rejected with `400 unknown_field` (strict schemas, to catch typos in integrations).

6.2 Error format

```
{
  "error": {
    "type": "invalid_request_error",
    "code": "too_many_stops",
    "message": "A route can have at most 5 stops.",
    "param": "stops",
    "request_id": "req_01J8…"
  }
}
```


+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

HTTP		When
400	`validation_error`, `unknown_field`, `too_many_stops`, `stop_without_parcels`, `over_van_capacity`, `outside_service_area`, `vehicle_below_requirement`, `unsupported_order_type`, `api_key_in_query`, `idempotency_key_required`	Bad input
401	`invalid_api_key`, `api_key_revoked`, `api_key_expired`	Auth
402	`insufficient_funds`	Wallet cannot cover the total
403	`insufficient_scope`, `ip_not_allowed`, `api_access_disabled`, `environment_mismatch`	Authz
404	`not_found`	Unknown or other-business resource
409	`duplicate_external_reference`, `price_exceeds_max`, `cancellation_not_allowed`, `idempotency_in_progress`, `already_cancelled`	State conflict
412	`business_not_approved`	Approval gate (body mirrors dashboard: `approval_status`, `reason`)
422	`address_not_found`, `idempotency_key_reused`	Semantically invalid
429	`rate_limited`, `daily_quota_exceeded`	Throttled (`Retry-After`)
500	`internal_error`	Unexpected (Sentry-captured, request id returned)
502	`payment_provider_error`, `routing_provider_error`	Upstream
503	`api_disabled`	Kill switch

6.3 Resource: Aerrand

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

```
{
  "id": "b7b6a4b2-1f0e-4a4c-9d3e-2c0d6d8a9f11",
  "object": "aerrand",
  "environment": "live",
  "test_mode": false,
  "status": "accepted",
  "external_reference": "PO-88213",
  "metadata": { "store_id": "yyz-01", "channel": "shopify" },
  "pickup": {
    "address": "120 Front St W, Toronto, ON M5J 2M3",
    "lat": 43.6446, "lng": -79.3806,
    "contact": { "name": "Acme Warehouse", "phone": "+14165550100", "email": "ops@acme.example" },
    "instructions": "Dock 3, ask for Sam"
  },
  "stops": [
    {
      "order_id": "b7b6a4b2-1f0e-4a4c-9d3e-2c0d6d8a9f11",
      "index": 0,
      "status": "accepted",
      "address": "55 Bloor St W, Toronto, ON",
      "lat": 43.6693, "lng": -79.3868,
      "recipient": { "name": "Jordan Lee", "phone": "+14165550199" },
      "instructions": "Buzz 1204",
      "dropoff_preference": "hand_to_recipient",
      "parcels": { "small": 2, "medium": 0, "large": 0, "xl": 0 },
      "delivered_at": null,
      "proof": null
    }
  ],
  "scheduled_at": null,
  "vehicle": "scooter",
  "pricing": {
    "currency": "CAD",
    "route_km": "4.12",
    "subtotal": "9.40",
    "hst": "1.22",
    "total": "10.62",
    "breakdown": { "dropBaseTotal": 8.0, "distanceCharge": 1.4, "sizeAddOnTotal": 0.0, "handlingTotal":
      0.0, "volumeSurcharge": 0.0, "recommendedVehicle": "scooter" }
  },
  "payment": { "mode": "wallet", "status": "paid", "reference": "wallet:wt_01J8…", "checkout_url": null
    },
  "driver": {
    "id": "66d0c1…", "name": "Alex", "phone_masked": "+1416•••0142",
    "vehicle": "scooter", "plate": "CFGH 123", "photo_url": "https://…",
    "location": { "lat": 43.65, "lng": -79.38, "heading": 90, "updated_at": "2026-08-29T14:10:05Z" }
  },
  "tracking_url": "https://business.aerrand.com/track/b7b6a4b2-1f0e-4a4c-9d3e-2c0d6d8a9f11",
  "batch_id": null,
  "cancellation": null,
  "timeline": [
    { "status": "searching", "at": "2026-08-29T14:03:01Z" },
    { "status": "accepted",  "at": "2026-08-29T14:05:40Z" }
  ],
  "created_at": "2026-08-29T14:03:00Z",
  "updated_at": "2026-08-29T14:05:40Z"
}
```


Public statuses (aerrand-level; per-stop statuses use the same vocabulary):

`pending_payment -> searching -> accepted -> arrived_at_pickup -> picked_up -> in_transit -> delivered`, terminal `cancelled`. For multi-stop, aerrand status = the least-advanced non-terminal stop; `delivered` only when every stop is delivered. Mapping from internal statuses is in Appendix C.

6.4 Endpoints

Method	Path	Scope	Idempotent	Purpose
GET	`/v1/me`	any	—	Key + business + approval + wallet summary

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

Method	Path	Scope	Idempotent	Purpose
POST	`/v1/quotes`	`quotes:read`	—	Price a route
GET	`/v1/quotes/{quote_id}`	`quotes:read`	—	Retrieve a quote
POST	`/v1/aerrands`	`aerrands:write`	required (live)	Create single/multi-stop aerrand
GET	`/v1/aerrands`	`aerrands:read`	—	List (cursor)
GET	`/v1/aerrands/{id}`	`aerrands:read`	—	Retrieve
GET	`/v1/aerrands/{id}/tracking`	`aerrands:read`	—	Live tracking
GET	`/v1/aerrands/{id}/proof`	`aerrands:read`	—	Proof of delivery (S)
POST	`/v1/aerrands/{id}/cancel`	`aerrands:cancel`	yes	Cancel
PATCH	`/v1/aerrands/{id}`	`aerrands:write`	yes	Pre-dispatch edits (C)
GET	`/v1/wallet`	`wallet:read`	—	Balance + transactions
GET/POST	`/v1/webhooks`	`webhooks:manage`	POST yes	Endpoints
GET/PATCH/DELETE	`/v1/webhooks/{id}`	`webhooks:manage`	—	Endpoint
POST	`/v1/webhooks/{id}/test`	`webhooks:manage`	—	Send test event
POST	`/v1/webhooks/{id}/rotate-secret`	`webhooks:manage`	—	Rotate secret
GET	`/v1/webhooks/{id}/deliveries`	`webhooks:manage`	—	Delivery log
GET	`/v1/events`	`aerrands:read`	—	Event history
POST	`/v1/events/{id}/redeliver`	`webhooks:manage`	—	Redeliver
POST	`/v1/test/aerrands/{id}/advance`	`aerrands:write` (test keys)	—	Simulate lifecycle (S)
GET	`/v1/health`	none	—	Liveness (no auth, for Uptime Kuma)
GET	`/v1/openapi.json`, `/docs`	none	—	Spec + Redoc

`POST /v1/quotes`

```
{
  "pickup": { "lat": 43.6446, "lng": -79.3806 },
  "stops": [
    { "lat": 43.6693, "lng": -79.3868, "parcels": { "small": 2 } },
    { "address": "1 Yonge St, Toronto, ON", "parcels": { "medium": 1, "large": 1 } }
  ]
}
```

Response `200`:

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

```
{
  "id": "qt_01J8N3…",
  "object": "quote",
  "expires_at": "2026-08-29T14:13:00Z",
  "currency": "CAD",
  "route_km": "7.90",
  "stop_count": 2,
  "total_parcels": 4,
  "subtotal": "37.63",
  "hst": "4.89",
  "total": "42.52",
  "recommended_vehicle": "car",
  "requires_cargo_vehicle": false,
  "total_cargo_units": "6.0",
  "breakdown": { "dropBaseTotal": 11.5, "distanceCharge": 6.13, "sizeAddOnTotal": 20.0, "handlingTotal":
    0.0, "volumeSurcharge": 0.0 },
  "resolved_stops": [ { "index": 1, "lat": 43.6429, "lng": -79.3776, "formatted_address": "1 Yonge St,
    Toronto, ON M5E 1W7" } ]
}
```

`POST /v1/aerrands`

Headers: `Authorization: Bearer …`, `Idempotency-Key: <uuid>`, `Content-Type: application/json`.

```
{
  "quote_id": "qt_01J8N3…",
  "external_reference": "PO-88213",
  "metadata": { "store_id": "yyz-01" },
  "pickup": {
    "address": "120 Front St W, Toronto, ON M5J 2M3",
    "lat": 43.6446, "lng": -79.3806,
    "contact": { "name": "Acme Warehouse", "phone": "+14165550100" },
    "instructions": "Dock 3, ask for Sam"
  },
  "stops": [
    {
      "address": "55 Bloor St W, Toronto, ON",
      "lat": 43.6693, "lng": -79.3868,
      "recipient": { "name": "Jordan Lee", "phone": "+14165550199" },
      "instructions": "Buzz 1204",
      "dropoff_preference": "hand_to_recipient",
      "parcels": { "small": 2 },
      "external_reference": "ORDER-1001"
    },
    {
      "address": "1 Yonge St, Toronto, ON",
      "recipient": { "name": "Front desk", "phone": "+14165550150" },
      "dropoff_preference": "reception",
      "parcels": { "medium": 1, "large": 1 }
    }
  ],
  "scheduled_at": null,
  "vehicle": "car",
  "max_total": "45.00",
  "payment": { "mode": "wallet" }
}
```

Response `201`: the Aerrand object (§6.3) with `status: "searching"`, `payment.status: "paid"`, `batch_id` set (2 stops).

`checkout` mode response adds `"payment": { "mode": "checkout", "status": "pending", "checkout_url": "https://checkout.stripe.com/c/pay/cs_…", "expires_at": "…" }` and `status: "pending_payment"`.

`POST /v1/aerrands/{id}/cancel`

```
{ "reason": "Customer cancelled order", "partial": false }
```

Response `200`:

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

```
{
  "id": "…", "status": "cancelled",
  "cancellation": {
    "from_stage": "AERRANDER_ASSIGNED",
    "fee": "4.00", "aerrander_compensation": "2.68",
    "refund": "38.52", "refund_destination": "wallet",
    "event_label": "B-03", "at": "2026-08-29T14:20:11Z"
  }
}
```

6.5 Webhook events

Event	Trigger (internal)	Writer
`aerrand.created`	rows created (any payment mode)	Django `create_aerrand`
`aerrand.paid`	`payment_status` -> true (wallet debit or Stripe webhook)	Django `_mark_order_paid_by_id`
`aerrand.searching`	status -> `broadcasted` (`_start_driver_search`)	Django
`aerrand.accepted`	driver accepted (delivery `status=accepted`, `driverId` set)	Node `deliveryController`
`aerrand.arrived_at_pickup`	delivery `status=arrived` / `workflow.phase=arrived_pickup`	Node
`aerrand.picked_up`	`picked_up` / `pickup_confirmed`	Node
`aerrand.in_transit`	`in_transit`	Node
`aerrand.stop_delivered`	a stop's delivery `status=delivered` (multi-stop)	Node
`aerrand.delivered`	last stop delivered	Node (dispatcher derives "last")
`aerrand.cancelled`	any cancel path (business, expiry, no-show, admin, driver)	Django / Node
`aerrand.payment_expired`	`expire_unpaid_api_aerrands`	Django
`wallet.balance_low` (C)	balance < threshold after a debit	Django

Example delivery:

```
POST https://integrator.example/aerrand/webhook
Aerrand-Signature: t=1756476611,v1=5f1a…c9
Aerrand-Event-Id: evt_01J8N4…
Aerrand-Event-Type: aerrand.accepted
Content-Type: application/json

{"id":"evt_01J8N4…","type":"aerrand.accepted","api_version":"2026-08-29","created":1756476611,"test_mode
  ":false,
 "data":{"object":{…Aerrand…},"previous_status":"searching"}}
```

Verification (Python):

```
import hmac, hashlib, time
def verify(secret: str, header: str, body: bytes, tolerance=300) -> bool:
    parts = dict(p.split("=", 1) for p in header.split(","))
    t = int(parts["t"]); sig = parts["v1"]
    if abs(time.time() - t) > tolerance: return False
    expected = hmac.new(secret.encode(), f"{t}.".encode() + body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, sig)
```


+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

7. Data model

All new collections live in the shared Mongo database (Django-owned; snake_case keys). Django model -> `db_table`.

7.1 `ApiKey` -> `api_keys`

Field	Type	Notes
`id`	UUID pk	`key_<uuid>` in API responses
`business_owner_id`	UUID, indexed	`CustomUser.id` of the owner
`created_by_id`	UUID	actor
`name`	str(64)	unique with `business_owner_id`, `environment`
`environment`	`live` | `test`	
`key_prefix`	str(24)	display only
`key_hash`	str(64), **unique index**	HMAC-SHA256 hex
`scopes`	list[str]	
`status`	`active` | `rotating` | `revoked`	
`expires_at`	datetime, nullable	
`ip_allowlist`	list[str]	CIDRs
`rate_limit_override`	JSON, nullable	`{per_minute, per_hour, create_per_minute, daily_quota}`
`last_used_at`, `last_used_ip`		coalesced writes
`created_at`, `revoked_at`, `revoked_by`, `rotated_from_id`		

7.2 `ApiQuote` -> `api_quotes`

`id` (`qt_…`), `business_owner_id`, `api_key_id`, `environment`, `inputs_hash` (SHA-256 of canonical pickup+stops), `inputs` (JSON), `route_km`, `breakdown` (JSON), `total` (decimal str), `resolved_stops`, `created_at`, `expires_at` (**TTL index**, `expireAfterSeconds=0`).

7.3 `IdempotencyRecord` -> `api_idempotency_keys`

`api_key_id` + `key` (**unique compound index**), `fingerprint`, `status` (`in_progress`/`completed`), `response_status`, `response_body` (JSON), `created_at` (**TTL 24 h**).

7.4 `WebhookEndpoint` -> `webhook_endpoints`

`id` (`we_…`), `business_owner_id` (indexed), `url`, `description`, `events` (list or `["*"]`), `environment`, `secret_encrypted`, `previous_secret_encrypted` + `previous_secret_expires_at` (rotation overlap), `status` (`enabled`/`disabled`), `disabled_reason`, `consecutive_failures`, `last_success_at`, `last_failure_at`, `created_at`, `updated_at`.

7.5 `WebhookOutbox` -> `webhook_outbox`

`id`, `event_id` (`evt_…`, unique), `event_type`, `order_id`, `batch_id`, `business_owner_id` (indexed), `channel`, `previous_status`, `test_mode`, `status` (`pending`/`claimed`/`dispatched`/`dropped`), `claimed_at`, `claimed_by`, `attempts`, `created_at` (indexed with `status` for the poll query), `dispatched_at`. Retention 30 days (TTL on `dispatched_at`).

7.6 `WebhookDelivery` -> `webhook_deliveries`

`id`, `endpoint_id` (indexed), `event_id` (indexed), `attempt`, `scheduled_at`, `sent_at`, `response_status`, `response_body` (<= 1 KB), `duration_ms`, `error`, `next_retry_at` (indexed, for the retry poll), `status` (`pending`/`succeeded`/`failed`/`exhausted`),

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

`created_at` (**TTL 30 d**).

7.7 `ApiRequestLog` -> `api_request_logs`

`request_id`, `api_key_id` (indexed), `business_owner_id`, `method`, `path`, `status`, `duration_ms`, `ip`, `idempotency_key`, `idempotent_replayed`, `error_code`, `created_at` (**TTL 7 d**). No bodies, no headers.

7.8 `ApiAuditLog` -> `api_audit_log`

`action` (`key.created`/`key.rotated`/`key.revoked`/`webhook.created`/…/`admin.api_access_disabled`), `business_owner_id`, `actor_id`, `actor_email`, `target_id`, `ip`, `user_agent`, `detail` (JSON), `created_at`. No TTL.

7.9 Changes to existing models

Model	Change
`order.Order`	`channel` gains value `"api"`; new `api_key_id` (str, nullable, indexed), `external_reference` (str(128), nullable; compound index `(customer_id, external_reference)`), `metadata` (JSON, nullable), `test_mode` (bool, default False, indexed), `payment_mode` (str(16), nullable).
`domain.Delivery` (mapper)	`channel="api"` passes through; new `origin` sub-doc `{source:"api", api_key_id, external_reference, test_mode}` written by `current_order_to_delivery_payload`. Node reads nothing new.
`custom_user.BusinessProfile`	`api_access_enabled` (bool, default True), `api_daily_quota` (int, nullable), `wallet_low_balance_threshold` (decimal, nullable).
`custom_user.WalletTransaction`	`TRANSACTION_TYPES` += `aerrand_charge`, `aerrand_refund`; new `order_id` (str, nullable, indexed), `reference` (str, nullable).

Indexes not expressible through Django's `Index` (TTL) are created in a `RunPython` migration step using the underlying pymongo collection, consistent with the existing `patch_idempotent_schema_editor` approach.

---

8. Non-functional requirements

ID	Requirement
NFR-01	**Latency**: p95 <= 400 ms for `POST /v1/aerrands` when `quote_id` is supplied (no routing call); <= 2.5 s without (<= 5 Google Distance Matrix legs). p95 <= 150 ms for reads. Measured at Caddy.
NFR-02	**Throughput**: 50 req/s aggregate on the current droplet without added replicas; creation <= 5/s sustained.
NFR-03	**Availability**: same as the stack (single droplet, autoheal). `GET /v1/health` monitored by Uptime Kuma; dispatcher exposes `GET /health` on an internal port for the compose healthcheck.
NFR-04	**Webhook latency**: event enqueued -> first delivery attempt <= 5 s p95.
NFR-05	**Observability**: Sentry (existing `SENTRY_DSN_API`) with tags `api_key_prefix`, `business_id`, `environment`; `Authorization` header scrubbed in `before_send`. Structured log line per request (json) with `request_id`.
NFR-06	**Data retention**: request logs 7 d, webhook deliveries/outbox 30 d, idempotency 24 h, quotes 10 min, audit log indefinite.
NFR-07	**Backward compatibility**: no change in behaviour for `channel="business"`/`"customer"` orders; all existing tests pass; Node driver backend changes are additive (outbox insert only, wrapped in try/catch, never blocks the transition).
NFR-08	**Portability**: no new managed services besides Redis; everything runs in the existing compose stack and CI matrix.
NFR-09	**Documentation**: every public field and error code documented in the OpenAPI spec; spec validated in CI (`npx @redocly/cli lint`).

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

9. Security requirements

ID	Requirement
SEC-01	Keys hashed with a server-side pepper (`API_KEY_HASH_PEPPER`, >= 32 random bytes). Rotating the pepper invalidates every key — documented as a break-glass procedure only.
SEC-02	Keys never appear in: Mongo (except hash), gunicorn/Caddy access logs (Caddy log format excludes `Authorization`), Sentry, `api_request_logs`, emails (prefix only), browser storage (dashboard shows once and does not persist).
SEC-03	TLS 1.2+ only on `api.aerrand.com`; HSTS `max-age=31536000`. HTTP -> HTTPS redirect.
SEC-04	Strict tenant isolation: every query in `developer.services` filters on `business_owner_id`/`customer_id` derived from the key — never from request parameters. Cross-tenant ids return 404. Covered by tests (SEC-T1).
SEC-05	Least privilege by scope; `webhooks:manage` and `aerrands:cancel` are separate from `aerrands:write` so a storefront key can create but not cancel.
SEC-06	Key creation/rotation/revocation from the dashboard requires a **fresh re-authentication** (password confirm within 5 min) — S.
SEC-07	Webhook SSRF controls per FR-WH-08; outbound requests use a dedicated `requests.Session` with no proxies, 10 s timeout, `allow_redirects=False`, response <= 64 KB read.
SEC-08	Webhook secrets encrypted at rest (Fernet); encryption key only in the root `.env`; rotation procedure documented.
SEC-09	Input hardening: strict serializers (`unknown_field`), string length caps, E.164 phones, https-only URLs, coordinate bounds, metadata caps, body size cap at Caddy (`request_body { max_size 256KB }`).
SEC-10	Rate limiting per key and per IP (FR-RATE-*); failed-auth per-IP limiter; Sentry alert on > 100 failed auths / 5 min from one IP.
SEC-11	Audit trail (FR-KEY-11, §7.8) and owner email on every key/webhook change.
SEC-12	PII minimisation: recipient names/phones are required for delivery but never included in logs or Sentry; `proof` URLs are signed and short-lived (reuse `custom_user.documents.sign_name` pattern, 60 min).
SEC-13	Legacy hardening note (out of scope but tracked): `GET /order/` (fetch-all) and other `AllowAny` listing endpoints remain reachable on `app.`/`business.aerrand.com`; the `/v1` design must not depend on them and a separate ticket should gate them.
SEC-14	Compliance: data stays in the existing DigitalOcean region; PIPEDA-aligned retention (§8 NFR-06); a business can request deletion of its API logs via support.

---

10. Implementation plan — codebase touch points

Paths are relative to `Aerrand_backen_infra/`.

10.1 Django `aerrand_api`

File	Change
`aerrand_api/settings.py`	Add `"developer"` to `INSTALLED_APPS`; `CACHES` (django-redis when `REDIS_URL`, LocMem fallback with warning); new settings read from env (Appendix D); Sentry `before_send` header scrubber; `MIGRATION_MODULES` unchanged. Leave `REST_FRAMEWORK` defaults as-is (v1 views declare their own classes).
`aerrand_api/urls.py`	`path('v1/', include('developer.urls_v1'))`; `path('docs', RedocV1View)`.
`developer/*`	New app per §4.3.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

File	Change
order/views.py	(a) Extract the body of _mark_order_paid_by_id into mark_order_paid(order, reference, source) and keep the id-based wrapper for the Stripe webhook. (b) Extract cancel_order internals into order/services.py::cancel_business_order(order, *, initiated_by, wait_minutes=None, driver_distance_km=None, return_incremental_km=None). (c) _resolve_stripe_key gains an explicit channel argument; create_payment_link gains success_url/cancel_url overrides. (d) orders_by_customer accepts ?channel=a,b. No behavioural change for existing callers.
order/models.py	Order fields per §7.9 + migration.
order/business_pricing.py	Unchanged (pure engine reused). Add VEHICLE_TIER_ORDER helper for FR-CREATE-07.
domain/mappers.py	current_order_to_delivery_payload: pass channel (already) and new origin sub-doc; delivery_to_order_response surfaces origin.
domain/services.py	DeliveryService.sync_from_order: early-return when order.test_mode.
notification/services.py	notify_order_status -> developer.services.webhooks.enqueue_from_order(order, status) (fail-safe, try/except).
custom_user/urls.py, custom_user/views.py	Register dashboard endpoints (FR-UI-08) implemented in developer/views_dashboard.py.
custom_user/models.py	BusinessProfile + WalletTransaction fields per §7.9.
order/admin_views.py, order/urls.py	Admin endpoints (FR-ADMIN-02) from developer/views_admin.py; admin_dashboard/report aggregates exclude test_mode.
requirements.txt	django-redis, redis, cryptography, phonenumbers.
docs/openapi-v1.json	New spec (FR-DOCS-01).
mongo_migrations/	n/a (app-local migrations under developer/migrations/).

10.2 Node aerrand_driver_backend

File	Change
services/webhookOutbox.js (new)	enqueue({ delivery, eventType, previousStatus }) -> db.collection('webhook_outbox').insertOne(...) when delivery.channel === 'api' or delivery.origin?.source === 'api'. Never throws (logs and returns).
controllers/deliveryController.js	Call enqueue right after each successful status write for accept, arrived, picked_up, in_transit, delivered, driver-initiated cancel — at the same points that push user_notifications.
docs/openapi.json	No change (driver API unchanged).

aerrand_customer_backend: no change (API aerrands are Django-origin orders; the customer backend's feed is not involved).

10.3 aerrand_business SPA

File	Change
src/app/pages/Profile.tsx	New developers section (FR-UI-01..06) — split into components/developers/{ApiKeysPanel,WebhooksPanel,ApiLogsPanel}.tsx.
src/app/services/api.ts	developerApi (keys, webhooks, logs); orderApi.byCustomer(customerId, "business,api").
src/app/pages/{Aerrands,OngoingAerrands,UpcomingAerrands,PastAerrands,DeliveryDetail}.tsx	"API" badge, external_reference, metadata, test-data toggle.
src/app/utils/business.ts	role helpers for FR-KEY-02.

10.4 aerrand_admin SPA

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

`src/pages/BusinessesPage.tsx` (API access panel), `src/pages/DashboardPage.tsx` (API overview card), `src/pages/OrdersPage.tsx` (channel filter).

10.5 Gateway and compose

File	Change
`Caddyfile`	New `api.aerrand.com` site block (below). `Caddyfile.dev`: `api.localhost` equivalent.
`docker-compose.yml`, `docker-compose.prod.yml`	Add `redis` (`redis:7-alpine`, `expose 6379`, `redis_data` volume, healthcheck `redis-cli ping`); add `aerrand-webhook-dispatcher` (same image as `aerrand-api`, `command: python manage.py run_webhook_dispatcher`, `env_file ./.env`, `depends_on: [redis]`, healthcheck).
`docker-compose.dev.yml`	Dev overrides for both.
`.github/workflows/deploy.yml`	No new image; the dispatcher reuses `aerrand-api:${IMAGE_TAG}`. Add `redocly lint` step for the spec.
`MONITORING.md`	Add `https://api.aerrand.com/v1/health` monitor + dispatcher container.
`README.md`	Routing table + API section.

```
# ------------------------------------------------------------------------------
# Public developer API (api.aerrand.com). Only /v1 and docs are exposed; the
# legacy Django routes are NOT reachable on this host. Point an A record at
# the droplet.
# ------------------------------------------------------------------------------
api.aerrand.com {
■encode gzip zstd
■header Strict-Transport-Security "max-age=31536000; includeSubDomains"
■header -Server
■request_body {
■■max_size 256KB
■}

■@public path /v1/* /docs /docs/* /health
■handle @public {
■■reverse_proxy aerrand-api:8000 {
■■■header_up X-Request-Id {http.request.uuid}
■■}
■}

■handle {
■■respond `{"error":{"type":"invalid_request_error","code":"not_found","message":"Not found"}}` 404
■}

■log {
■■output stdout
■■format json {
■■■# never log the Authorization header
■■}
■}
}
```

10.6 DNS

Add `api.aerrand.com` A record -> droplet IP in DigitalOcean (same as `business.`/`admin.`). Caddy provisions the certificate on first request.

---

11. Configuration and deployment

- All new settings are read from the shared root `.env` (Appendix D). Remember the two known `.env` pitfalls in this stack: inline `# comments` and trailing whitespace on a value break it, and `docker compose watch` does **not** re-read `.env` — use `--force-recreate --build`.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

- `API_KEY_HASH_PEPPER` and `WEBHOOK_SECRET_ENCRYPTION_KEY` must be generated once (`python -c "import secrets;print(secrets.token_urlsafe(48))"` / `python -c "from cryptography.fernet import Fernet;print(Fernet.generate_key().decode())"`) and **backed up** with the same convention as the Firebase rules backups — losing either invalidates all keys / all webhook secrets.
- Deploy order: Django (`aerrand-api` + dispatcher, migrations run at container start) -> Node driver backend (outbox writes are harmless before the dispatcher exists) -> business SPA -> admin SPA. Nothing requires a coordinated cutover.
- Feature flags: `API_KEYS_ENABLED` (global), `BusinessProfile.api_access_enabled` (per business), `API_WEBHOOKS_ENABLED` (dispatcher no-op when false).

12. Testing and acceptance criteria

12.1 Automated tests (Django; run in Docker with `MONGOURI_PROD` overridden to a local Mongo, per the existing test workflow)

ID	Test
T-KEY-1	Generate/hash/verify round-trip; revoked/expired/rotating states; prefix search; 10-key cap.
T-AUTH-1	Bearer + `X-Api-Key` accepted; DRF Token rejected on `/v1`; API key rejected on `/order/`. Scope matrix per endpoint. Environment mismatch -> 404. IP allowlist.
T-ELIG-1	412 for `awaiting_documents`/`pending`/`rejected`; approved and legacy(null) pass; `api_access_enabled=false` -> 403.
T-QUOTE-1	**Parity**: for 50 randomised routes, `POST /v1/quotes.total == POST /order/business/route/estimate/.chargeTotal`. Stop cap, van cap, service area.
T-CREATE-1	Single & 5-stop creation: row counts, batch fields, per-stop price sum == total (cent reconciliation), `size`/`volume`/`vehicle_type` mapping, inline instructions, `channel="api"`, delivery sync shape (`pending_payment` vs `broadcasted`).
T-CREATE-2	Atomicity: inject failure on stop 3 -> zero rows remain, wallet untouched.
T-CREATE-3	`quote_id` reuse (no routing call), expired quote -> re-price, `max_total` conflict, duplicate `external_reference`.
T-PAY-1	Wallet debit race: 20 concurrent creates against a balance covering 5 -> exactly 5 succeed, 15 × 402, balance == 0, 5 `aerrand_charge` rows.
T-PAY-2	Checkout mode: business Stripe key selected with **no Origin header** (mock `stripe.checkout.Session.create`, assert `api_key`), webhook marks all siblings paid, expiry sweep cancels + emits event.
T-CANCEL-1	Fee/refund per stage (reuse `test_business_cancellation` fixtures), all-or-nothing vs `partial`, wallet refund transaction.
T-IDEM-1	Replay, fingerprint mismatch (422), concurrent (409), 5xx release.
T-RATE-1	Limits + headers with Redis (compose) and LocMem fallback.
T-WH-1	Signature vector tests; SSRF rejections (127.0.0.1, 10.x, 169.254.169.254, `metadata.google.internal`); retry schedule; auto-disable after 7 d; rotation overlap signs with both secrets.
T-WH-2	Dispatcher: claim/lease semantics, per-aerrand ordering, `stop_delivered` vs `delivered` derivation for batches, drop when no endpoint.
T-SEC-1	Tenant isolation: key A cannot read/cancel/track key B's aerrand (404), including by `external_reference`.
T-TEST-1	Test keys: no delivery sync, no debit, no Stripe, excluded from admin aggregates; `advance` emits events.
T-COMPAT-1	Entire existing suite (`order.tests`, `test_business_*`, `test_coupons`, `domain.tests`, `custom_user.*`) passes unchanged.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

Node: Jest unit test for `services/webhookOutbox.js` (insert shape, never throws) and a controller test asserting an insert on accept for `channel="api"` and none for `channel="customer"`.

12.2 Manual / E2E (staging)

1. Approve a pilot business in admin -> create `aer_test_` and `aer_live_` keys in the dashboard; confirm audit rows + owner email.
2. Top up wallet with Stripe test card; `POST /v1/aerrands` (wallet) -> aerrand appears in the dashboard with "API" badge, driver app feed shows the job, driver accepts -> `aerrand.accepted` arrives at a webhook receiver within 10 s with a valid signature; deliver -> `aerrand.delivered`; wallet transaction visible.
3. `checkout` mode -> pay on the hosted page -> webhook path -> `aerrand.paid`/`aerrand.searching`.
4. Cancel at `AERRANDER_ASSIGNED` -> fee `4.00`, refund to wallet, `aerrand.cancelled`.
5. Revoke the key -> immediate 401; rotate -> old key works for the grace window only.
6. Rate-limit and idempotency behaviour verified with a script (replay returns identical body + `Idempotent-Replayed: true`).
7. Uptime Kuma monitor green for `/v1/health`; Sentry shows scrubbed headers on a forced error.

12.3 Acceptance criteria (definition of done)

- All FR-* marked **M** implemented and covered by the tests above; CI green including spec lint.
- Pricing parity test passes; no client-supplied amounts influence charges on `/v1`.
- Pen-test checklist §9 reviewed; no key material found in any log/Sentry event during E2E.
- Docs published at `https://api.aerrand.com/docs`; quickstart completed by someone outside the core team in < 30 min.
- Two pilot businesses live for 2 weeks with webhook success rate >= 99% and zero cross-tenant incidents.

---

13. Rollout plan

Phase	Scope	Exit criteria
0 — Foundations (1–2 wks)	Redis + dispatcher services, Caddy host + DNS, `developer` app skeleton, `Order`/`Wallet`/`BusinessProfile` migrations, refactors in `order/views.py` (paid path, cancel path, Stripe key resolution), pepper/encryption keys provisioned.	Existing suite green on staging; `GET /v1/health` monitored.
1 — Private beta (2–3 wks)	Keys (create/revoke/rotate), `/v1/me`, quotes, create (wallet + checkout), get/list/tracking, cancel, idempotency, rate limits, dashboard Developers section (keys only), admin panel, docs. `API_KEYS_ENABLED=true`, `api_access_enabled` on 2–3 pilot businesses only.	Acceptance §12.3 items 1–4, 6.
2 — Webhooks & test mode (2 wks)	Outbox writers (Django + Node), dispatcher delivery/retries, endpoint management (API + dashboard), event history/redeliver, test-mode `advance`, Postman collection.	Webhook success >= 99% over 1 wk; item 2 of E2E.
3 — General availability	`api_access_enabled` default on for all approved businesses; marketing page on `aerrand.com/developers`; SDKs (C).	Support runbook written; on-call knows the kill switches.
4 — Payments expansion	`card_on_file` (Stripe SetupIntent/off-session), auto-refill wallet, `wallet.balance_low`; Stripe reconciliation for checkout-mode cancellations.	Separate TRD.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

Rollback: set `API_KEYS_ENABLED=false` (all `/v1` -> 503) and stop the dispatcher; no data migration needs reversing (new fields are nullable; new collections are additive).

14. Open questions and decisions required

ID	Question	Recommendation
OQ-1	Channel value for API aerrands: new `"api"` vs reuse `"business"`.	`"api"` (this spec). Requires the dashboard filter change (FR-UI-07) and a grep of Node/Flutter for `channel === 'business'` assumptions before Phase 1 ends.
OQ-2	Default payment mode: wallet (prepaid) vs card-on-file first.	**Wallet** — zero new Stripe surface, race-safe, and the wallet/top-up flow already exists. Card-on-file in Phase 4.
OQ-3	Should API aerrands honour scheduled delayed dispatch? Today a paid standard order is broadcast immediately regardless of `date_time`.	Accept `scheduled_at` <= 7 days but document "broadcast on payment" until a dispatch scheduler exists (tracked separately; `Delivery.dispatch_at` already exists for Guard).
OQ-4	Stripe reconciliation on cancellation (capture fee / release remainder) is unimplemented for checkout-paid orders.	Keep checkout mode in v1 but document that cancellation fees for checkout-paid aerrands are recorded, not settled, until Phase 4. Wallet mode settles fully.
OQ-5	Business pricing constants live in code (`business_pricing.DEFAULT_CONFIG`), unlike the customer engine's DB-backed `PricingConfig`. Exposing them externally raises the cost of changes.	Add a `BusinessPricingConfig` singleton (mirroring `order.models.PricingConfig`) in Phase 0 or 1 so price changes don't require a deploy.
OQ-6	Service-area enforcement (`API_SERVICE_AREA_BBOX`) — GTA only?	Enable with a GTA bounding polygon at GA; off during beta.
OQ-7	Key ownership when the creating `business_admin` leaves the company.	Keys are owner-scoped (FR-KEY-01) so nothing breaks; the audit log records the creator. Owner is emailed on every change.
OQ-8	`batch_size=1` semantics: always stamp `batch_id` for API aerrands (uniform id) vs only for N>1.	**Only for N>1** (FR-CREATE-03) to avoid any driver-app regression; the API resolves either id.
OQ-9	Is `STRIPE_SECRET_KEY_BUSINESS` moving to a live key before GA?	Required for checkout mode in production (FR-PAY-08). Decide with finance.
OQ-10	Commercial terms: is API usage metered/billed, and what is the fair-use limit?	Not needed for beta; the per-business quota fields (FR-RATE-04) leave room.
OQ-11	Legacy `AllowAny` order endpoints on `app./business.aerrand.com` (SEC-13).	Open a separate hardening ticket; not a blocker for `/v1`.

Appendix A — Glossary

Term	Meaning
Aerrand	A business delivery job: one pickup and 1–5 drop-off stops. Internally one `Order` row per stop (plus `batch_*` for N>1) mirrored to one or more `deliveries` docs.
Aerrander	A driver.
Business owner	`CustomUser` with `account_type="business"` and a `BusinessProfile`; team members are `business_<role>` linked by `invited_by`.
Channel	`Order.channel` / `Delivery.channel`: `customer`, `business`, or (new) `api`.
Outbox	`webhook_outbox` collection — the queue of lifecycle events awaiting webhook dispatch.
Pepper	Server-side secret mixed into the key hash so a database leak alone cannot verify keys.
Test mode	`aer_test_` keys / `Order.test_mode=true`: no drivers, no money.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

Appendix B — cURL quickstart

```
export AERRAND_KEY=aer_test_…

# 1. Who am I / can I book?
curl -s https://api.aerrand.com/v1/me -H "Authorization: Bearer $AERRAND_KEY"

# 2. Quote
curl -s https://api.aerrand.com/v1/quotes \
  -H "Authorization: Bearer $AERRAND_KEY" -H "Content-Type: application/json" \
  -d '{"pickup":{"lat":43.6446,"lng":-79.3806},
       "stops":[{"lat":43.6693,"lng":-79.3868,"parcels":{"small":2}}]}'

# 3. Create (idempotent)
curl -s https://api.aerrand.com/v1/aerrands \
  -H "Authorization: Bearer $AERRAND_KEY" -H "Content-Type: application/json" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{"quote_id":"qt_…","external_reference":"PO-1",
       "pickup":{"address":"120 Front St W, Toronto","lat":43.6446,"lng":-79.3806,
                 "contact":{"name":"Acme","phone":"+14165550100"}},
       "stops":[{"address":"55 Bloor St W, Toronto","lat":43.6693,"lng":-79.3868,
                 "recipient":{"name":"Jordan","phone":"+14165550199"},
                 "parcels":{"small":2},"dropoff_preference":"hand_to_recipient"}],
       "payment":{"mode":"wallet"}}'

# 4. Track
curl -s https://api.aerrand.com/v1/aerrands/<id>/tracking -H "Authorization: Bearer $AERRAND_KEY"

# 5. Register a webhook
curl -s https://api.aerrand.com/v1/webhooks \
  -H "Authorization: Bearer $AERRAND_KEY" -H "Content-Type: application/json" \
  -d '{"url":"https://example.com/aerrand/webhook","events":["*"]}'
```

Appendix C — Status mapping

Public status	Internal / delivery status (canonical)	Delivery
`pending_payment`	`draft`, `pending_payment` (unpaid)	—
`searching`	`pending` (paid), `broadcasted`	—
`accepted`	`accepted`, `assigned`	`assigned`
`arrived_at_pickup`	`arrived`	`arrived_pickup`
`picked_up`	`picked_up`	`pickup_confirmed`
`in_transit`	`in_transit`, `in-transit`, `in_progress`	`in_transit`
`delivered`	`delivered`, `completed`	`completed`
`cancelled`	`cancelled`	`cancelled`

Business lifecycle stages (`Order.lifecycle_stage`, used by the cancellation fee engine) are exposed as `cancellation.from_stage` only.

Appendix D — Environment variable reference

Variable	Required	Default	Purpose
`API_KEYS_ENABLED`	no	`true`	Global kill switch for `/v1`.
`API_KEY_HASH_PEPPER`	**yes**	—	HMAC pepper for key hashing (>= 32 bytes). Back it up.
`API_PUBLIC_BASE_URL`	no	`https://api.aerrand.com`	Used in docs, `topup_url`, emails.
`API_RATE_LIMIT_DEFAULT`	no	`60/min,3000/hour`	Per-key default.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com

Variable	Required	Default	Purpose
`API_RATE_LIMIT_CREATE`	no	`20/min`	`POST /v1/aerrands`.
`API_RATE_LIMIT_QUOTE`	no	`120/min`	`POST /v1/quotes`.
`API_DAILY_QUOTA_DEFAULT`	no	`2000`	Aerrands/day per business.
`API_IDEMPOTENCY_TTL_HOURS`	no	`24`	
`API_QUOTE_TTL_SECONDS`	no	`600`	
`API_UNPAID_TTL_HOURS`	no	`24`	Checkout-mode expiry.
`API_SERVICE_AREA_BBOX`	no	unset	`minLat,minLng,maxLat,maxLng`; unset = no check.
`API_WEBHOOKS_ENABLED`	no	`true`	Dispatcher no-op when false.
`WEBHOOK_SECRET_ENCRYPTION_KEY`	**yes** (Phase 2)	—	Fernet key for endpoint secrets. Back it up.
`WEBHOOK_DISPATCHER_POLL_SECONDS`	no	`2`	
`WEBHOOK_TIMEOUT_SECONDS`	no	`10`	
`WEBHOOK_MAX_ATTEMPTS`	no	`6`	
`REDIS_URL`	recommended	unset	`redis://redis:6379/0`; LocMem fallback with warning.
Reused: `STRIPE_SECRET_KEY_BUSINESS`, `STRIPE_WEBHOOK_SECRET_BUSINESS`, `AERRAND_BUSINESS_WEB_URL`, `GOOGLE_MAPS_API_KEY`, `MONGOURI_PROD`, `RESEND_*`, `SENTRY_DSN_API`, `FIREBASE_*`			Existing behaviour.

End of document.

+1 (226) 773-7563
111 Riverside Drive, Windsor, ON N9A 2S6, Canada
operations@aerrand.com
www.aerrand.com